



slington college
(इस्लिङ्टन कलेज)

CC5051NI - Databases

60% Individual Coursework

2025-26 Spring

Credit: 15 Semester Long Module

Student Name: Alina B.K.

London Met ID: 24058871

College ID: NP01CP4S250039

Assignment Due Date: Sunday, January 18, 2026

Assignment Submission Date: Sunday, January 18, 2026

Word Count: 6,578

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

CC5051 Databases Final Coursework Submission

 Islington College,Nepal

Document Details

Submission ID

trn:oid:::3618:126426562

Submission Date

Jan 18, 2026, 3:25 PM GMT+5:45

Download Date

Jan 18, 2026, 3:46 PM GMT+5:45

File Name

Alina B.K. 2058871.pdf

File Size

1.8 MB

57 Pages

6,375 Words

45,352 Characters

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Quoted Text
- Small Matches (less than 8 words)

Exclusions

- 4 Excluded Sources
- 83 Excluded Matches

Match Groups

-  **64 Not Cited or Quoted** 13%
Matches with neither in-text citation nor quotation marks
-  **7 Missing Quotations** 1%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 0%  Publications
- 14%  Submitted works (Student Papers)

Match Groups

-  **64 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
 -  **7 Missing Quotations 1%**
Matches that are still very similar to source material
 -  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
 -  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks
-

Top Sources

- 1%  Internet sources
- 0%  Publications
- 14%  Submitted works (Student Papers)

Table of Contents

1	Introduction	1
1.1	Introduction of Business and Strategic Strengths	1
1.2	Current Business Activities	2
1.3	Business Operations	3
1.4	Business Rule	4
1.5	Assumptions.....	4
2	The initial Entity Relationship Model.....	5
2.1	Identification of Entities and Attributes	5
2.2	Initial Entity Relationships Diagram of the Entities with attributes and relationships	8
3	Normalization	9
3.1	Un-Normalized Form (UNF)	9
3.2	First Normalized Form (1NF).....	10
3.3	Second Normalized Form (2NF)	11
3.4	Third Normalized Form (3NF).....	14
3.5	Final Relations in Third Normal Form (3NF)	17
4	Data Dictionary and Final ERD.....	18
4.1	List of Entities After Normalization and Attributes	18
4.2	Final Entity Relationship Diagram (ERD).....	21
5	Implementation	22
5.1	Create Tables.....	23
5.2	Adding Rows to Tables.....	28
6	Database Querying.....	39
6.1	Information Query.....	39
6.2	Transaction Query	42

7	Critical Evaluation	45
7.1	Critical Evaluation of module, its usage and relation with other subjects	45
7.2	Critical Assessment of coursework.....	46
8	Drop Query and Database Dump File Creation	47
8.1	Drop Query	47
8.2	Database Dump File Creation	49
9	Conclusion.....	50
10	References	51

Table of Figures

Figure 1: Logo of SajiloYatra	1
Figure 2: SajiloYatra	1
Figure 3: Initial ERD	8
Figure 4: Final Entity Relationship Diagram	21
Figure 5: Logging as a Admin	22
Figure 6: Creating a New User	22
Figure 7: Granting Privileges to New User	22
Figure 8: Logging as a User	22
Figure 9: Creating Passenger Table	23
Figure 10: Creating Routing Table	24
Figure 11: Creating Bus Table	24
Figure 12: Creating Staff Table	25
Figure 13: Creating Trip Table	25
Figure 14: Creating Booking Table	26
Figure 15: Creating Ticket Table	26
Figure 16: Creating Payment Table	27
Figure 17: Creating Passenger_Trip Table	27
Figure 18: Inserting Values in Passenger Table	28
Figure 19: Displaying Passenger Table	28
Figure 20: Inserting Values in Route Table	29
Figure 21: Displaying Route Table	29
Figure 22: Inserting Values in Bus Table	30
Figure 23: Displaying Route Table	30
Figure 24: Inserting Values in Staff Table	31
Figure 25: Displaying Staff Table	31
Figure 26: Inserting Values in Trip Table	32
Figure 27: Displaying Trip Table	32
Figure 28: Inserting Values in Booking Table	33
Figure 29: Displaying Booking Table	34
Figure 30: Inserting Values in Ticket Table	35
Figure 31: Displaying Ticket Table	36
Figure 32: Inserting Values in Payment Table	37
Figure 33: Displaying Payment Table	37
Figure 34: Inserting Values in Passenger_Trip Table	38
Figure 35: Displaying Passenger_Trip Table	38
Figure 36: List all routes along with total number of trips	39
Figure 37: Trips departing from Pokhara	40
Figure 38: Passengers whose name starts with A	40
Figure 39: Passengers who booked more than 2 trips in December 2025	41
Figure 40: Tickets issued in January 2025 with payment status	41
Figure 41: The trip with the latest departure time	42
Figure 42: Three passengers who booked the most trips	42
Figure 43: Passengers who paid more than average payment	43
Figure 44: Total revenue generated per route	43
Figure 45: Trips with less than 50% booking capacity	44
Figure 46: Dropped Passenger Table	47

Figure 47: Dropped Route Table	47
Figure 48: Dropped Bus Table.....	47
Figure 49: Dropped Staff Table	47
Figure 50: Dropped Trip Table	47
Figure 51: Dropped Booking Table	48
Figure 52: Dropped Ticket Table.....	48
Figure 53: Dropped Payment Table.....	48
Figure 54: Dropped Passenger_Trip Table	48
Figure 55: Drop Query and Database Dump File Creation	49

Table of Tables

Table 1: Identifications of entities and attributes (Passenger)	5
Table 2: Identifications of entities and attributes (Trip)	6
Table 3: Identifications of entities and attributes (Route)	6
Table 4: Identifications of entities and attributes (Bus)	7
Table 5: Identifications of entities and attributes (Staff)	7
Table 6: Identifications of entities and attributes (Booking)	7
Table 7: Identifications of entities and attributes (Ticket)	8
Table 8: Identifications of entities and attributes (Payment)	8
Table 9: Passenger Table	18
Table 10: Route Table	18
Table 11: Bus Table	19
Table 12: Staff Table	19
Table 13: Trip Table	20
Table 14: Booking Table	20
Table 15: Ticket Table	20
Table 16: Payment Table	21
Table 17: Passenger-Inbox Table	21

1 Introduction

1.1 Introduction of Business and Strategic Strengths

SajiloYatra is a newly rising inter-city transport service launched in Nepal with a goal of enhancing transport connectivity between major cities like Kathmandu, Pokhara, Chitwan, and Butwal. It was launched by Mr. Kiran Thapa and has the aim of overcoming problems associated with the current inter-city transport service, which include inefficient physical booking processes, problems related to overbooking, absence of a central database of passengers, and a lack of coordination in transport buses and routes.



Figure 1: Logo of SajiloYatra



Figure 2: SajiloYatra

The strength of Sajilo Yatra lies in its plan of implementing a modern, centralized, and technology-based system of booking and management. The proposed system allows for efficient management of journey, routes, buses, passengers, tickets, as well as payments, all within one integrated system. This technology-based system is known to work towards increased efficiency, minimized human errors, as well as increased customer satisfaction.

Another key strength of SajiloYatra is its flexibility and scalability. The software can handle different types of bookings, including Regular, Premium, and VIP. Therefore, the solution can help the business serve different types of customers. Moreover, the software can handle aspects that could delay the journey, including rescheduling trips, handling bus unavailability due to maintenance, and handling payments. Such strengths align with the goals of making SajiloYatra a trusted competitor in the provision of intercity travel services in Nepal. (Mohtashim, 2025)

1.2 Current Business Activities

To provide consistent and dependable inter-city transportation services, SajiloYatra engages in many different activities and processes. Each of these activities is accomplished using a database-based system, as follows:

1.2.1 Intercity Route Development

SajiloYatra is dedicated to finding and building routes for intercity travel that link popular cities such as Kathmandu, Pokhara, Chitwan, and Butwal.

1.2.2 Service Planning and Scheduling

The company will offer travel services based on the frequencies of travel and the timing of the journey as planned based on demand.

1.2.3 Customer service

The service also allows booking of tickets by passengers traveling in different classes like Regular, Premium, and VIP. This is offered by this company.

1.2.4 Revenue Generation Activities

The company earns revenue through ticket sales. Additionally, it is responsible for ticket fare management for various routes.

1.2.5 Business Growth and Expansion

The purpose of the company is to extend its operations and services with new routes, trips, and buses and increase efficiency with an online system.

1.3 Business Operations

1.3.1 Online Trip Booking Management

The system can record on-line bookings where it stores information such as passenger details, trips chosen, booking class, time of booking, and ticket status.

1.3.2 Passenger and Ticket Processing

Every passenger is given a unique identity, and tickets are printed based on confirmed bookings. It is ensured that every ticket is assigned to a particular passenger and a particular journey.

1.3.3 Transport/Bus Seat Allocation

The buses are allocated routes, and the seating capacity is considered to avoid overbooking. The available seats are automatically updated after each booking.

1.3.4 Staff Assignment and Monitoring

Drivers and conductors are allocated routes, while staff information is also managed to facilitate safe and regulated trip operations.

1.3.5 Payment Processing and Tracking

The system can record payment information such as amount, date, and type. The status regarding payment of tickets is checked to determine if it is paid or unpaid.

1.3.6 Operations Updates & Schedule Control

Schedules for trips and status updates are done in the system in case there are delays, route deviations, or if buses are not available due to maintenance purposes.

1.3.7 I/O Accelerator Control and Integrity

The database operates within business rules, relationships, and constraints to ensure consistency, accuracy, and reliability of operational data.

1.4 Business Rule

It defines operational parameters and relationships regarding the creation, storage, and management of data in the SajiloYatra database system also how intercity travel service operations to ensure the proper handling and to maintain consistent data.

- A passenger can book zero, one, or many tickets.
Each ticket must be booked by one passenger.
- A ticket is issued for exactly one trip only.
Each trip can have zero, one, or many tickets.
- A passenger can travel on one or many trips through ticket bookings.
Each trip can be travelled by one or many passengers.
- Each trip must be scheduled on exactly one route.
A route can have one or many trips.
- Each trip must be operated by exactly one bus.
A bus can operate zero, one, or many trips over time.
- The number of tickets issued for a trip must not exceed the bus seating capacity.
- Each ticket may have zero or one payment.
Each payment is linked to exactly one ticket.
- A passenger can receive messages related to trips or tickets.

1.5 Assumptions

- A passenger must be registered before booking a ticket.
- A ticket cannot exist without a passenger and a trip.
- Payments are recorded only after a ticket has been issued.
- A bus cannot exceed its seating capacity under any condition.
- A trip cannot operate without at least one staff member assigned.
- Messages are used only for system and trip-related notifications.

2 The initial Entity Relationship Model

The (ER Model) is a type of conceptual (Logical) database that is typically used to design a database. In the ER model, the entities from the "real world" and how they relate to each other are the focus of the design. The ER Initial Model concentrates on Passengers, Trips, Routes, Bus, Staff, Booking, Ticket, Payment, Messages, and the Passenger-Inbox, describing how these entities are interlinked in the SajiloYatra system. (Jain S. , 2025)

- **Entity:** An entity is an object or item that exists in the "real" world and is represented in a relational database using a single table, which can be defined as a single building block of a relational database system. Example: Passenger, Route, Trip.
- **Attribute:** Attributes are the characteristics that are used to describe an entity type. In an ERD, attributes are depicted as ovals. Example: PassengerID, IssueDate.
- **Relationship:** A relationship type is a correlation or link between two or more entity types. Example: A Passenger books a Ticket.

2.1 Identification of Entities and Attributes

It is the properties associated with the entity and identifying entities that are part of the SajiloYatra system. There are three main types of entities that will form the basis for the development of an Initial ERD.

1. Passenger

S. No.	Attribute Name	Data Type	Size	Constraint
1	passenger_id	Number	10	Primary Key
2	first_name	Character	30	
3	last_name	Character	30	
4	phone_number	Number	10	
5	email	Character	50	
6	date_of_birth	Date	-	
7	gender	Character	10	
8	address	Character	100	
9	registration_date	Date	-	
10	passenger_status	Character	15	

Table 1: Identifications of entities and attributes (Passenger)

2. Trip

S. N0.	Attribute Name	Data Type	Size	Constraint
1	trip_id	Number	10	Primary Key
2	trip_date	Date	-	
3	departure_time	Character	-	
4	arrival_time	Character	-	
5	route_id	Number	10	
6	bus_id	Number	10	
7	trip_status	Character	15	
8	available_seats	Number	5	
9	total_seats	Number	5	
10	delay_reasons	Character	100	

Table 2: Identifications of entities and attributes (Trip)

3. Route

S. No.	Attribute Name	Data Type	Size	Constraint
1	route_id	Number	10	Primary Key
2	route_name	Character	50	
3	origin_city	Character	30	
4	destination_city	Character	30	
5	distance_km	Number	10	
6	estimated_duration	Number	5	
7	route_status	Character	15	
8	route_type	Character	20	
9	toll_required	Character	5	
10	created_date	Date	-	

Table 3: Identifications of entities and attributes (Route)

4. Bus

S. No.	Attribute Name	Data Type	Size	Constraint
1	bus_id	Number	10	Primary Key
2	bus_number	Character	20	
3	bus_type	Character	20	
4	seating_capacity	Number	3	
5	bus_status	Character	15	

*Table 4: Identifications of entities and attributes (Bus)***5. Staff**

S. No.	Attribute Name	Data Type	Size	Constraint
1	staff_id	Number	10	Primary Key
2	first_name	Character	30	
3	last_name	Character	30	
4	phone_number	Number	15	
5	role	Character	20	
6	Staff_status	Character	15	

*Table 5: Identifications of entities and attributes (Staff)***6. Booking**

S. No.	Attribute Name	Data Type	Size	Constraint
1	booking_id	Number	10	Primary Key
2	passenger_id	Number	30	
3	trip_id	Number	30	
4	boooking_date	Date	15	
5	booking_status	Character	20	

Table 6: Identifications of entities and attributes (Booking)

7. Ticket

S. No.	Attribute Name	Data Type	Size	Constraint
1	ticket_id	Number	10	Primary Key
2	booking_id	Number	10	
3	passenger_id	Number	30	
4	trip_id	Number	30	
5	fare	Number	8	
6	issue_date	Date	–	
7	ticket_status	Character	15	

Table 7: Identifications of entities and attributes (Ticket)

8. Payment

S. No.	Attribute Name	Data Type	Size	Constraint
1	payment_id	Number	10	Primary Key
2	ticket_id	Number	10	
3	amount	Number	8	
4	payment_date	Date	–	
5	payment_method	Character	20	
6	payment_status	Character	15	

Table 8: Identifications of entities and attributes (Payment)

2.2 Initial Entity Relationships Diagram of the Entities with attributes and relationships

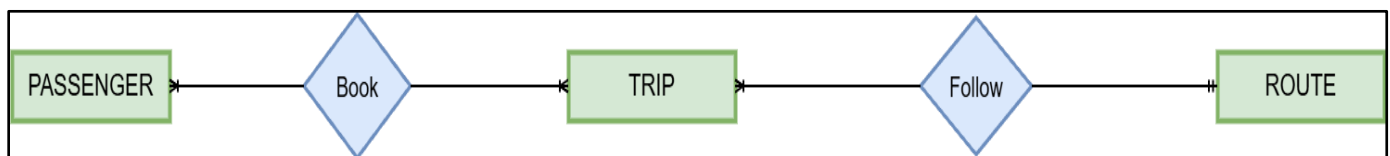


Figure 3: Initial ERD

3 Normalization

It is the process is also referred to as database normalization. Database normalization is very critical to the overall design of a Database. The benefits of the normalization process include improved database performance; improved database reliability through increased consistency and accuracy; easier to manage and maintain Databases; and improved ability to meet the changing demands of a business. (Jain S. , 2025).

- Database normalization aims to eliminate or reduce redundant data or copies of the same information found in multiple locations.
- Data redundancy is not only wasteful of space and storage resources but also creates problems with inconsistencies when updating, deleting or inserting data into a database.

3.1 Un-Normalized Form (UNF)

The simplest form of a Database Model is the UNF (Un-Normalized Form). This form of database will have the issue of duplicate data within the tables hence it won't be as efficient as a normalized database.

UNF databases are repeating groups and multiple valued attributes which create both redundancy & anomalies.

UNF: Here all the entities and attributes are collected with various repeating groups.

Passenger(passenger_id, first_name, last_name, phone_number, email, date_of_birth, gender, address, registration_date, passenger_status, {trip_id, trip_date, departure_time, arrival_time, trip_status, available_seats, total_seats, delay_reasons, route_id, route_name, origin_city, destination_city, distance_km, estimated_duration, route_status, route_type, toll_required, created_date}, {bus_id, bus_number, bus_type, seating_capacity, bus_status, {staff_id, first_name, last_name, phone_number, role, staff_status}}, {booking_id, booking_date, booking_status, ticket_id, fare, issue_date, ticket_status, payment_id, amount, payment_date, payment method, payment_status}))

3.2 First Normalized Form (1NF)

The first step in the process of reducing duplication, increasing the integrity of data, and reducing the occurrence of anomalies in the design of a relational database.

1NF: To accomplish this, repeating groups have been separated out, and the Primary Key has been propagated through each type of entity.

Addressing the repeating group in PASSENGER-U, we have:

PASSENGER

(PassengerID, FirstName, LastName, PhoneNumber, Email, DateOfBirth, Gender, Address, RegistrationDate, PassengerStatus)

PASSENGER-TRIP

(PassengerID, TripID, TripDate, DepartureTime, ArrivalTime, TripStatus, RouteID, RouteName, OriginCity, DestinationCity, DistanceKM, EstimatedDuration, RouteStatus, RouteType, TollRequired, AvailableSeats, TotalSeats, DelayReasons)

PASSENGER_BUS

(PassengerID, BusID, BusNumber, BusType, SeatingCapacity, BusStatus, StaffID, StaffFirstName, StaffLastName, StaffPhoneNumber, Role, StaffStatus)

PASSENGER_BOOKING

(PassengerID, BookingID, BookingDate, BookingStatus, TicketID, Fare, IssueDate, TicketStatus, PaymentID, Amount, PaymentDate, PaymentMethod, PaymentStatus)

3.3 Second Normalized Form (2NF)

A relation is in 2NF if the following conditions are met it satisfies the requirements of 1NF, and no partial dependency exists. That means every non-prime attribute (non-key attribute) in this case, must be dependent on the entire primary key.

2NF: Eliminate partial dependencies (attributes that depend on part of a composite key). The table should already be in 1NF.

Analysis of PASSENGER-TRIP

Composite Primary Key

PassengerID, TripID

Full Functional Dependencies (FFD)

$\text{TripID} \rightarrow \text{TripDate, DepartureTime, ArrivalTime, TripStatus}$

There is no full functional dependency in PASSENGER-TRIP because non-key attributes do not depend on the entire composite key (PassengerID, TripID).

Partial Functional Dependencies (PFD)

$\text{TripID} \rightarrow \text{TripDate, DepartureTime, ArrivalTime, TripStatus}$

Decomposition Results (to remove PFDs)

PASSENGER-TRIP

(PassengerID*, TripID*)

TRIP

(TripID, TripDate, DepartureTime, ArrivalTime, TripStatus, RouteID)

After decomposition, all non-key attributes are fully dependent on their primary keys.

Therefore, the relations are now in 2NF.

Analysis of PASSENGER-BOOKING**Composite Primary Key**

PassengerID, BookingID, TripID

Full Functional Dependencies (FFD)

BookingID $\rightarrow$ BookingDate, BookingStatus

There is no full functional dependency because non-key attributes do not depend on the entire composite key.

Partial Functional Dependencies (PFD)

BookingID $\rightarrow$ BookingDate, BookingStatus

Since BookingDate and BookingStatus depend only on BookingID, a partial dependency exists.

Decomposition Results**PASSENGER-BOOKING**

(PassengerID*, BookingID*, TripID*)

BOOKING

(BookingID, BookingDate, BookingStatus)

This removes the partial dependency and satisfies 2NF.

Analysis of PASSENGER-TICKET**Composite Primary Key**

PassengerID, TripID, TicketID

Full Functional Dependencies (FFD)

TicketID $\rightarrow$ Fare, IssueDate, TicketStatus

There is no full functional dependency because non-key attributes do not depend on the entire composite key.

Partial Functional Dependencies (PFD)

TicketID $\rightarrow$ Fare, IssueDate, TicketStatus

Partial dependency exists.

Decomposition Results**PASSENGER-TICKET**

(PassengerID*, TripID*, TicketID*)

TICKET

(TicketID, Fare, IssueDate, TicketStatus)

Now all non-key attributes depend on the full primary key. Relation is in 2NF.

Analysis of PASSENGER-PAYMENT**Composite Primary Key**

TicketID, PaymentID

Full Functional Dependencies (FFD)

TicketID, PaymentID $\rightarrow$ Amount, PaymentDate, PaymentMethod, PaymentStatus

Partial Functional Dependencies (PFD)

PaymentID $\rightarrow$ Amount, PaymentDate, PaymentMethod, PaymentStatus

Partial dependency exists.

Decomposition Results**PASSENGER-PAYMENT**

(TicketID*, PaymentID*)

PAYMENT

(PaymentID, TicketID, Amount, PaymentDate, PaymentMethod, PaymentStatus)

This satisfies 2NF.

3.4 Third Normalized Form (3NF)

A relation is said to be in 3NF if it is in 2NF and, furthermore, there are no transitive dependencies. In other words, the non-prime attributes must not be dependent on other non-prime attributes.

3NF: Eliminate transitive dependencies (non-key attributes depending on other non-key attributes).

Table must already be in 2NF.

Analysis of PASSENGER

Functional Dependency:

PassengerID $\rightarrow$ FirstName, LastName, PhoneNumber, Email, DateOfBirth, Gender, Address, RegistrationDate, PassengerStatus

All non-key attributes depend directly and only on PassengerID.

There is no dependency between non-key attributes.

So, PASSENGER is already in 3NF.

Analysis of TRIP

Functional Dependency:

TripID $\rightarrow$ TripDate, DepartureTime, ArrivalTime, TripStatus, RouteID

Here, RouteID is a foreign key, not a descriptive attribute.

TripDate, DepartureTime, ArrivalTime, and TripStatus depend directly on TripID.

There is no transitive dependency inside TRIP.

Then, TRIP is in 3NF.

Analysis of ROUTE

Functional Dependency:

RouteID $\rightarrow$ RouteName, OriginCity, DestinationCity, DistanceKM, EstimatedDuration, RouteType

All non-key attributes depend only on RouteID.

No non-key attribute determines another non-key attribute.

Hence, ROUTE is in 3NF.

Analysis of BUS**Functional Dependency:**

BusID $\rightarrow$ BusNumber, BusType, SeatingCapacity, BusStatus

All attributes are fully dependent on BusID.

No transitive dependency exists.

Therefore, BUS is in 3NF.

Analysis of STAFF**Functional Dependency:**

StaffID $\rightarrow$ StaffFirstName, StaffLastName, StaffPhoneNumber, Role, StaffStatus

All non-key attributes depend only on StaffID.

There is no dependency between non-key attributes.

Thus, STAFF is in 3NF.

Analysis of BOOKING**Functional Dependency:**

BookingID $\rightarrow$ BookingDate, BookingStatus

All attributes depend directly on BookingID.

No transitive dependency exists.

Accordingly, BOOKING is in 3NF.

Analysis of TICKET**Functional Dependency:**

TicketID $\rightarrow$ Fare, IssueDate, TicketStatus

Fare, IssueDate, and TicketStatus depend only on TicketID.

There is no non-key determining another non-key.

Therefore, TICKET is in 3NF.

Analysis of PAYMENT**Functional Dependency:**

PaymentID → TicketID, Amount, PaymentDate, PaymentMethod, PaymentStatus

Although TicketID appears, it is a foreign key, not a transitive dependency.

All descriptive attributes depend directly on PaymentID.

So, PAYMENT is in 3NF.

Analysis of Relationship Tables**PASSENGER-TRIP**

Primary Key: PassengerID, TripID

This table contains only key attributes.

Relations with only key attributes are automatically in 3NF.

PASSENGER-BOOKING

Primary Key: PassengerID, BookingID, TripID

Contains only identifiers, no descriptive attributes.

Thus, already in 3NF.

PASSENGER-TICKET

Primary Key: PassengerID, TripID, TicketID

No non-key attributes exist.

Hence, in 3NF.

PASSENGER-PAYMENT

Primary Key: TicketID, PaymentID

Contains only keys.

Then, in 3NF.

3.5 Final Relations in Third Normal Form (3NF)

After completing the normalization process till Third Normal Form (3NF) and removing all the repeating groups, partial dependencies, and transitive dependencies, the final relations of the Sajilo Yatra database system would be as follows:

PASSENGER

(PassengerID, FirstName, LastName, DateOfBirth, Email, PhoneNumber, Gender, Address, RegistrationDate, PassengerStatus)

ROUTE

(RouteID, RouteName, OriginCity, DestinationCity, DistanceKM, EstimatedDuration, RouteType, RouteStatus, TollRequired)

BUS

(BusID, BusNumber, BusType, SeatingCapacity, BusStatus)

STAFF

(StaffID, StaffFirstName, StaffLastName, StaffPhoneNumber, Role, StaffStatus)

TRIP

(TripID, TripDate, DepartureTime, ArrivalTime, TripStatus, CreatedDate, RouteID*, BusID*)

BOOKING

(BookingID, BookingDate, BookingStatus, PassengerID*, TripID*)

TICKET

(TicketID, Fare, IssueDate, TicketStatus, BookingID*)

PAYMENT

(PaymentID, Amount, PaymentDate, PaymentMethod, PaymentStatus, TicketID*)

PASSENGER_TRIP

(PassengerID, TripID)

Composite Primary Key: (PassengerID, TripID)

4 Data Dictionary and Final ERD

4.1 List of Entities After Normalization and Attributes

Entity 1: Passenger

Storing information of the passenger who has registered and traveled.

S.No.	Attribute Name	Data Type	Size	Constraint
1	PassengerID	Number	10	PRIMARY KEY
2	FirstName	Character	50	NOT NULL
3	LastName	Character	50	NOT NULL
4	DateOfBirth	Date	-	NOT NULL
5	Email	Character	100	UNIQUE
6	PhoneNumber	Character	15	NOT NULL
7	Gender	Character	10	NOTNULL
8	Address	Character	255	NOTNULL
9	RegistrationDate	Date	-	NOTNULL
10	PassengerStatus	Character	20	NOT NULL

Table 9: Passenger Table

Entity 2: Route

Storing information about the routes covered by buses.

S.No.	Attribute Name	Data Type	Size	Constraint
1	RouteID	Number	10	PRIMARY KEY
2	RouteName	Character	100	NOT NULL
3	OriginCity	Character	50	NOT NULL
4	DestinationCity	Character	50	NOT NULL
5	DistanceKM	Number	10	NOT NULL
6	EstimatedDuration	Number	5	NOT NULL
7	RouteType	Character	20	NOT NULL
8	RouteSttatus	Character	20	NOT NULL
9	TollRequired	Character	3	NOT NULL

Table 10: Route Table

Entity 3: Bus

This function records the planned trips on a given route, including the departure and arrival times and the assigned bus.

S.No.	Attribute Name	Data Type	Size	Constraint
1	BusID	Number	10	PRIMARY KEY
2	BusNumber	Character	20	NOT NULL
3	BusType	Character	30	NOT NULL
4	SeatingCapacity	NUMBER	3	NOT NULL
5	BusStatus	Character	20	NOT NULL

Table 11: Bus Table

Entity 4: Staff

This function stores information about the staff members assigned to buses, including their names, contact details, roles, and current status.

S.No.	Attribute Name	Data Type	Size	Constraint
1	StaffID	Number	10	PRIMARY KEY
2	StaffFirstName	Character	50	NOT NULL
3	StaffLastName	Character	50	NOT NULL
4	StaffPhoneNumber	Character	15	NOT NULL
5	Role	Character	20	NOT NULL
6	StaffStatus	Character	20	NOT NULL

Table 12: Staff Table

Entity 5: Trip

Storing details about trips.

S.No.	Attribute Name	Data Type	Size	Constraint
1	TripID	Number	10	PRIMARY KEY
2	TripDate	Date	-	NOT NULL
3	DepartureTime	Character	10	NOT NULL
4	ArrivalTime	Character	10	NOT NULL
5	TripStatus	Character	20	NOT NULL
6	CreatedDate	Date	-	NOT NULL
7	RouteID	Number	10	FOREIGN KEY
8	BusID	Number	10	FOREIGN KEY

Table 13: Trip Table

Entity 6: Booking

This function records the bookings made by passengers for a specific trip, including the booking date, status, and the associated passenger and trip.

S.No.	Attribute Name	Data Type	Size	Constraint
1	BookingID	Number	10	PRIMARY KEY
2	BookingDate	Date	-	NOT NULL
3	BookingStatus	Character	20	NOT NULL
4	PassengerID	Number	10	FOREIGN KEY
5	TripID	Number	10	FOREIGN KEY

Table 14: Booking Table

Entity 7: Ticket

Holds tickets issued for passengers on journeys, allowing for charging and date of issue.

S.No.	Attribute Name	Data Type	Size	Constraint
1	TicketID	Number	10	PRIMARY KEY
2	Fare	Number	8	NOT NULL
3	IssueDte	Date	-	NOT NULL
4	TicketStatus	Character	20	NOT NULL
5	BookingID	Number	10	FOREIGN KEY

Table 15: Ticket Table

Entity 6: Payment

Manages payments done for ticket purchases based on amount, date, and type.

S.No.	Attribute Name	Data Type	Size	Constraint
1	PaymentID	Number	10	PRIMARY KEY
2	Amount	Number	8	NOT NULL
3	PaymentDate	Date	-	NOT NULL
4	PaymentMethod	Character	20	NOT NULL
5	PaymentStatus	Character	20	NOT NULL
6	TicketID	Number	10	FOREIGN KEY

Table 16: Payment Table

Entity 8: Passenger_Trip

This function records which passengers are assigned to which trips, mapping passengers to their planned trips..

S.No.	Attribute Name	Data Type	Size	Constraint	Composite Constraint
1	PassengerID	Number	10	FOREIGN KEY	PRIMARY KEY
2	TripID	Number	10	FOREIGN KEY	

Table 17: Passenger-Inbox Table

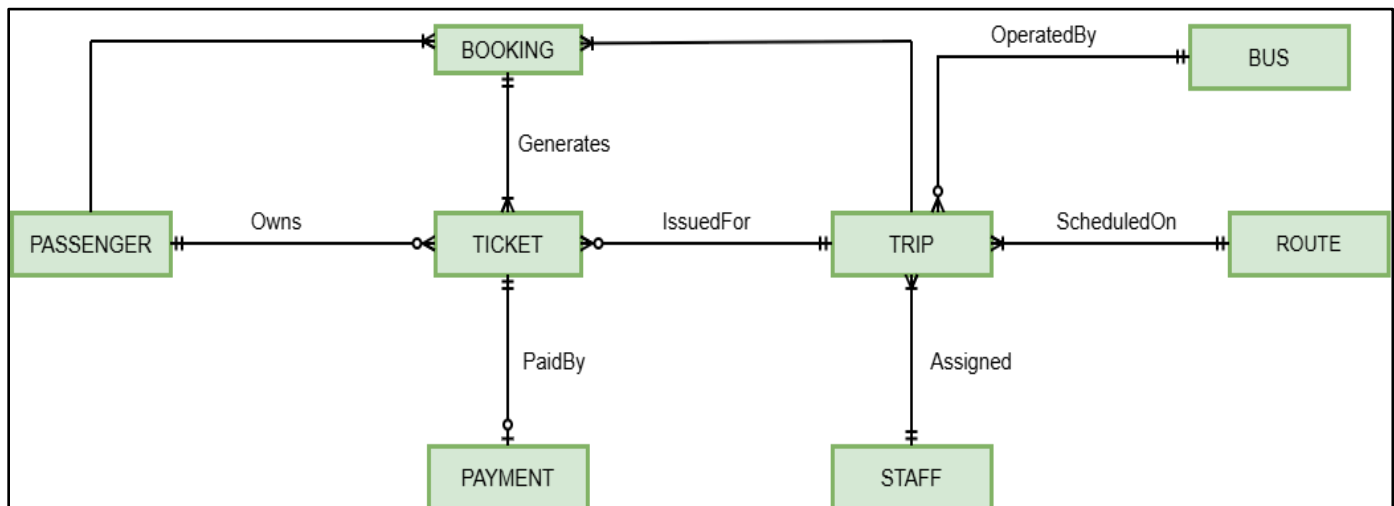
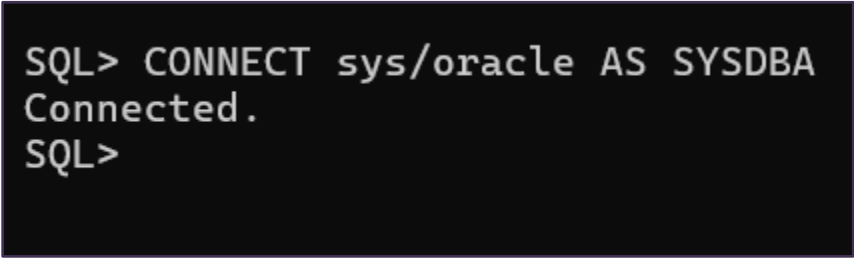
4.2 Final Entity Relationship Diagram (ERD)

Figure 4: Final Entity Relationship Diagram

5 Implementation

The implementation phase includes designing the structure of the database and managing the data using Oracle SQL. During this phase, users, tables, and constraints are designed using SQL commands. Passengers tables, including other tables, are designed with appropriate data types, primary keys, and constraints to ensure data integrity is maintained.

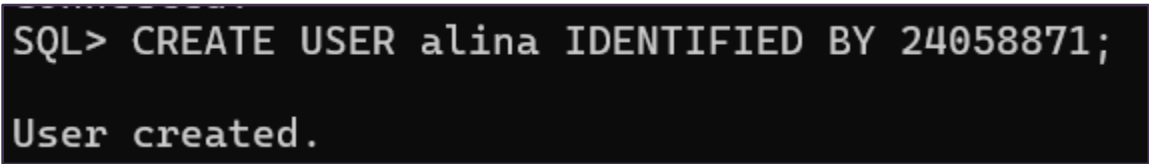
1. Log in as Admin



```
SQL> CONNECT sys/oracle AS SYSDBA
Connected.
SQL>
```

Figure 5: Logging as a Admin

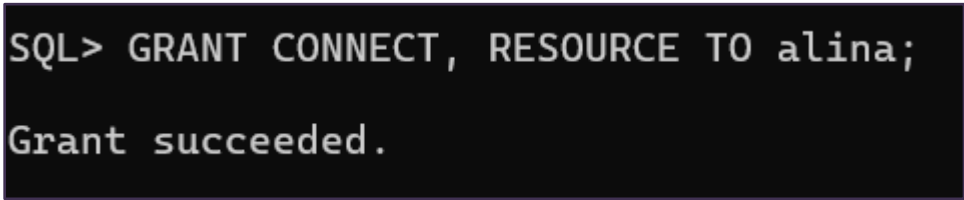
2. Create New User



```
SQL> CREATE USER alina IDENTIFIED BY 24058871;
User created.
```

Figure 6: Creating a New User

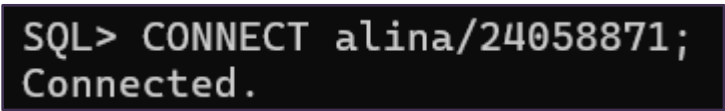
3. Granting Privileges to New User



```
SQL> GRANT CONNECT, RESOURCE TO alina;
Grant succeeded.
```

Figure 7: Granting Privileges to New User

4. Login as a User



```
SQL> CONNECT alina/24058871;
Connected.
```

Figure 8: Logging as a User

5.1 Create Tables

5. Create Table Passenger

```
SQL> CONNECT alina/24058871;
Connected.
SQL> Create TABLE Passenger (
  2 PassengerID NUMBER(10) PRIMARY KEY,
  3 FirstName CHARACTER(50) NOT NULL,
  4 LastName CHARACTER(50) NOT NULL,
  5 DateOfBirth DATE NOT NULL,
  6 Email CHARACTER(100) UNIQUE,
  7 PhoneNumber CHARACTER(15) NOT NULL,
  8 Gender CHARACTER(10) NOT NULL,
  9 Address CHARACTER(255) NOT NULL,
 10 RegistrationDate DATE NOT NULL,
 11 PassengerStatus CHARACTER(20) NOT NULL
 12 );

Table created.

SQL> desc Passenger;
Name                                     Null?    Type
-----
PASSENGERID                             NOT NULL NUMBER(10)
FIRSTNAME                               NOT NULL CHAR(50)
LASTNAME                                NOT NULL CHAR(50)
DATEOFBIRTH                             NOT NULL DATE
EMAIL                                     NOT NULL CHAR(100)
PHONENUMBER                             NOT NULL CHAR(15)
GENDER                                  NOT NULL CHAR(10)
ADDRESS                                 NOT NULL CHAR(255)
REGISTRATIONDATE                        NOT NULL DATE
PASSENGERSTATUS                         NOT NULL CHAR(20)
```

Figure 9: Creating Passenger Table

6. Create Table Route

```
SQL> CREATE TABLE Route(
 2 RouteID NUMBER(10) PRIMARY KEY,
 3 RouteName CHARACTER(100) NOT NULL,
 4 OriginCity CHARACTER(50) NOT NULL,
 5 DestinationCity CHARACTER(50) NOT NULL,
 6 DistanceKM NUMBER(10) NOT NULL,
 7 EstimatedDuration NUMBER(5) NOT NULL,
 8 RouteType CHARACTER(20) NOT NULL,
 9 RouteStatus CHARACTER(20) NOT NULL,
10 TollRequired CHARACTER(3) NOT NULL
11 );
```

Table created.

```
SQL> desc Route;
```

Name	Null?	Type
ROUTEID	NOT NULL	NUMBER(10)
ROUTENAME	NOT NULL	CHAR(100)
ORIGINCITY	NOT NULL	CHAR(50)
DESTINATIONCITY	NOT NULL	CHAR(50)
DISTANCEKM	NOT NULL	NUMBER(10)
ESTIMATEDDURATION	NOT NULL	NUMBER(5)
ROUTETYPE	NOT NULL	CHAR(20)
ROUTESTATUS	NOT NULL	CHAR(20)
TOLLREQUIRED	NOT NULL	CHAR(3)

Figure 10: Creating Routing Table

7. Create Table Bus

```
SQL> CREATE TABLE Bus (
 2 BusID NUMBER(10) PRIMARY KEY,
 3 BusNumber CHARACTER(20) NOT NULL,
 4 BusType CHARACTER(30) NOT NULL,
 5 SeatingCapacity NUMBER(3) NOT NULL,
 6 BusStatus CHARACTER(20) NOT NULL
 7 );
```

Table created.

```
SQL> desc Bus;
```

Name	Null?	Type
BUSID	NOT NULL	NUMBER(10)
BUSNUMBER	NOT NULL	CHAR(20)
BUSTYPE	NOT NULL	CHAR(30)
SEATINGCAPACITY	NOT NULL	NUMBER(3)
BUSSTATUS	NOT NULL	CHAR(20)

Figure 11: Creating Bus Table

8. Create Table Staff

```
SQL> CREATE TABLE Staff (
  2  StaffID NUMBER(10) PRIMARY KEY,
  3  StaffFirstName CHARACTER(50) NOT NULL,
  4  StaffLastName CHARACTER(50) NOT NULL,
  5  StaffPhoneNumber CHARACTER(15) NOT NULL,
  6  Role CHARACTER(20) NOT NULL,
  7  StaffStatus CHARACTER(20) NOT NULL
  8  );
```

Table created.

```
SQL> desc Staff;
```

Name	Null?	Type
STAFFID	NOT NULL	NUMBER(10)
STAFFFIRSTNAME	NOT NULL	CHAR(50)
STAFFLASTNAME	NOT NULL	CHAR(50)
STAFFPHONENUMBER	NOT NULL	CHAR(15)
ROLE	NOT NULL	CHAR(20)
STAFFSTATUS	NOT NULL	CHAR(20)

Figure 12: Creating Staff Table

9. Create Table Trip

```
SQL> CREATE TABLE Trip (
  2  TripID NUMBER(10) PRIMARY KEY,
  3  TripDate DATE NOT NULL,
  4  DepartureTime CHARACTER(10) NOT NULL,
  5  ArrivalTime CHARACTER(10) NOT NULL,
  6  TripStatus CHARACTER(20) NOT NULL,
  7  RouteID NUMBER(10),
  8  BusID NUMBER(10),
  9  FOREIGN KEY (RouteID) REFERENCES Route(RouteID),
  10 FOREIGN KEY (BusID) REFERENCES Bus(BusID)
  11 );
```

Table created.

```
SQL> desc Trip;
```

Name	Null?	Type
TRIPID	NOT NULL	NUMBER(10)
TRIPDATE	NOT NULL	DATE
DEPARTURETIME	NOT NULL	CHAR(10)
ARRIVALTIME	NOT NULL	CHAR(10)
TRIPSTATUS	NOT NULL	CHAR(20)
ROUTEID		NUMBER(10)
BUSID		NUMBER(10)

Figure 13: Creating Trip Table

10. Create Table Booking

```
SQL> CREATE TABLE Booking (
  2  BookingID NUMBER(10) PRIMARY KEY,
  3  BookingDate DATE NOT NULL,
  4  BookingStatus CHARACTER(20) NOT NULL,
  5  PassengerID NUMBER(10),
  6  TripID NUMBER(10),
  7  FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID),
  8  FOREIGN KEY (TripID) REFERENCES Trip(TripID)
  9  );
```

Table created.

```
SQL> desc Booking;
```

Name	Null?	Type
BOOKINGID	NOT NULL	NUMBER(10)
BOOKINGDATE	NOT NULL	DATE
BOOKINGSTATUS	NOT NULL	CHAR(20)
PASSENGERID		NUMBER(10)
TRIPID		NUMBER(10)

Figure 14: Creating Booking Table

11. Create Table Ticket

```
SQL> CREATE TABLE Ticket (
  2  TicketID NUMBER(10) PRIMARY KEY,
  3  Fare NUMBER(8) NOT NULL,
  4  IssueDate DATE NOT NULL,
  5  TicketStatus CHARACTER(20) NOT NULL,
  6  BookingID NUMBER(10),
  7  FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
  8  );
```

Table created.

```
SQL> desc Ticket;
```

Name	Null?	Type
TICKETID	NOT NULL	NUMBER(10)
FARE	NOT NULL	NUMBER(8)
ISSUEDATE	NOT NULL	DATE
TICKETSTATUS	NOT NULL	CHAR(20)
BOOKINGID		NUMBER(10)

Figure 15: Creating Ticket Table

12. Create Table Payment

```
SQL> CREATE TABLE Payment (
  2 PaymentID NUMBER(10) PRIMARY KEY,
  3 Amount NUMBER(8) NOT NULL,
  4 PaymentDate DATE NOT NULL,
  5 PaymentMethod CHARACTER(20) NOT NULL,
  6 PaymentStatus CHARACTER(20) NOT NULL,
  7 TicketID NUMBER(10),
  8 FOREIGN KEY (TicketID) REFERENCES Ticket(TicketID)
  9 );
```

Table created.

```
SQL> desc Payment;
```

Name	Null?	Type
PAYMENTID	NOT NULL	NUMBER(10)
AMOUNT	NOT NULL	NUMBER(8)
PAYMENTDATE	NOT NULL	DATE
PAYMENTMETHOD	NOT NULL	CHAR(20)
PAYMENTSTATUS	NOT NULL	CHAR(20)
TICKETID		NUMBER(10)

Figure 16: Creating Payment Table

13. Create Table Passenger_Trip

```
SQL> CREATE TABLE Passenger_Trip (
  2 PassengerID NUMBER(10),
  3 TripID NUMBER(10),
  4 PRIMARY KEY (PassengerID, TripID),
  5 FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID),
  6 FOREIGN KEY (TripID) REFERENCES Trip(TripID)
  7 );
```

Table created.

```
SQL> desc Passenger_Trip;
```

Name	Null?	Type
PASSENGERID	NOT NULL	NUMBER(10)
TRIPID	NOT NULL	NUMBER(10)

Figure 17: Creating Passenger_Trip Table

5.2 Adding Rows to Tables

14. Add Rows in Passenger Table

I. Insert and Commit Command

```
SQL> INSERT INTO Passenger VALUES(1, 'Samikshya', 'Parajuli', DATE '1998-05-14', 'samikshya@gmail.com', '9817567483', 'Female', 'Kathmandu', DATE '2025-01-01', 'Active');
1 row created.

SQL> INSERT INTO Passenger VALUES(2, 'Czan', 'Thapa', DATE '1995-09-20', 'czan@gmail.com', '9845765036', 'Male', 'Pokhara', DATE '2025-01-02', 'Active');
1 row created.

SQL> INSERT INTO Passenger VALUES(3, 'Rebika', 'Pun', DATE '2000-02-11', 'rebika@gmail.com', '9854357622', 'Female', 'Chitwan', DATE '2025-01-03', 'Active');
1 row created.

SQL> INSERT INTO Passenger VALUES(4, 'Yoongi', 'Sunar', DATE '1997-07-01', 'yoongi@gmail.com', '98266647587', 'Male', 'Butwal', DATE '2025-01-04', 'Active');
1 row created.

SQL> INSERT INTO Passenger VALUES(5, 'Aakriti', 'Gurung', DATE '1999-11-22', 'aakriti@gmail.com', '9845782314', 'Female', 'Dharan', DATE '2025-01-05', 'Active');
1 row created.

SQL> INSERT INTO Passenger VALUES(6, 'Newson', 'Ghimire', DATE '1996-03-18', 'newson@gmail.com', '9823678954', 'Male', 'Lalitpur', DATE '2025-01-06', 'Active');
1 row created.

SQL> INSERT INTO Passenger VALUES(7, 'Deep', 'Magar', DATE '2001-08-09', 'deep@gmail.com', '9825000648', 'Female', 'Bhaktapur', DATE '2025-02-07', 'Active');
1 row created.

SQL> COMMIT;
Commit complete.
```

Figure 18: Inserting Values in Passenger Table

II. Select Command

```
SQL> SELECT * FROM Passenger;
```

PASSENGERID	FIRSTNAME	LASTNAME	DATEOFBIRTH	EMAIL	PHONENUMBER	GENDER	ADDRESS	REGISTRATION	PASSENGERS
1	Samikshya	Parajuli	14-MAY-98	samikshya@gmail.com	9817567483	Female	Kathmandu	01-JAN-25	Active
2	Czan	Thapa	20-SEP-95	czan@gmail.com	9845765036	Male	Pokhara	02-JAN-25	Active
3	Rebika	Pun	11-FEB-00	rebika@gmail.com	9854357622	Female	Chitwan	03-JAN-25	Active
4	Yoongi	Sunar	01-JUL-97	yoongi@gmail.com	98266647587	Male	Butwal	04-JAN-25	Active
5	Aakriti	Gurung	22-NOV-99	aakriti@gmail.com	9845782314	Female	Dharan	05-JAN-25	Active
6	Newson	Ghimire	18-MAR-96	newson@gmail.com	9823678954	Male	Lalitpur	06-JAN-25	Active
7	Deep	Magar	09-AUG-01	deep@gmail.com	9825000648	Female	Bhaktapur	07-FEB-25	Active

```
7 rows selected.
```

Figure 19: Displaying Passenger Table

15. Add Rows in Route Table

I. Insert and Commit Command

```
SQL> INSERT INTO Route VALUES(101, 'Kathmandu-Pokhara', 'Kathmandu', 'Pokhara', 200, 7, 'Highway', 'Active', 'Yes');
1 row created.

SQL> INSERT INTO Route VALUES(102, 'Kathmandu-Chitwan', 'Kathmandu', 'Chitwan', 160, 5, 'Highway', 'Active', 'No');
1 row created.

SQL> INSERT INTO Route VALUES(103, 'Pokhara-Butwal', 'Pokhara', 'Butwal', 180, 6, 'Highway', 'Active', 'Yes');
1 row created.

SQL> INSERT INTO Route VALUES(104, 'Kathmandu-Dharan', 'Kathmandu', 'Dharan', 390, 10, 'Long', 'Active', 'Yes');
1 row created.

SQL> INSERT INTO Route VALUES(105, 'Butwal-Chitwan', 'Butwal', 'Chitwan', 120, 4, 'Highway', 'Active', 'No');
1 row created.

SQL> INSERT INTO Route VALUES(106, 'Lalitpur-Pokhara', 'Lalitpur', 'Pokhara', 205, 7, 'Highway', 'Active', 'Yes');
1 row created.

SQL> INSERT INTO Route VALUES(107, 'Bhaktapur-KTM', 'Bhaktapur', 'Kathmandu', 15, 1, 'City', 'Active', 'No');
1 row created.

SQL> COMMIT;
Commit complete.
```

Figure 20: Inserting Values in Route Table

II. Select Command

```
SQL> SELECT * FROM Route;
```

ROUTEID	ROUTENAME	ORIGINCITY	DESTINATIONCITY	DISTANCEKM	ESTIMATEDDURATION	ROUTETYPE	ROUTESTATUS	TOLLR
101	Kathmandu-Pokhara	Kathmandu	Pokhara	200	7	Highway	Active	Yes
102	Kathmandu-Chitwan	Kathmandu	Chitwan	160	5	Highway	Active	No
103	Pokhara-Butwal	Pokhara	Butwal	180	6	Highway	Active	Yes
104	Kathmandu-Dharan	Kathmandu	Dharan	390	10	Long	Active	Yes
105	Butwal-Chitwan	Butwal	Chitwan	120	4	Highway	Active	No
106	Lalitpur-Pokhara	Lalitpur	Pokhara	205	7	Highway	Active	Yes
107	Bhaktapur-KTM	Bhaktapur	Kathmandu	15	1	City	Active	No

```
7 rows selected.
```

Figure 21: Displaying Route Table

16. Add Rows in Bus Table

I. Insert and Commit Command

```
SQL> INSERT INTO Bus VALUES (201, 'BA-1-KHA-1234', 'AC Deluxe', 40, 'Available');
1 row created.

SQL> INSERT INTO Bus VALUES (202, 'BA-2-KHA-2345', 'Non-AC', 45, 'Available');
1 row created.

SQL> INSERT INTO Bus VALUES (203, 'BA-3-KHA-3456', 'VIP', 30, 'Available');
1 row created.

SQL> INSERT INTO Bus VALUES (204, 'BA-4-KHA-4567', 'AC', 42, 'Available');
1 row created.

SQL> INSERT INTO Bus VALUES (205, 'BA-5-KHA-5678', 'Tourist', 35, 'Available');
1 row created.

SQL> INSERT INTO Bus VALUES (206, 'BA-6-KHA-6789', 'Sleeper', 28, 'Available');
1 row created.

SQL> INSERT INTO Bus VALUES (207, 'BA-7-KHA-7890', 'Mini', 25, 'Available');
1 row created.

SQL> COMMIT;

Commit complete.
```

Figure 22: Inserting Values in Bus Table

II. Select Command

```
SQL> SELECT * FROM Bus;
```

BUSID	BUSNUMBER	BUSTYPE	SEATINGCAPACITY	BUSSTATUS
201	BA-1-KHA-1234	AC Deluxe	40	Available
202	BA-2-KHA-2345	Non-AC	45	Available
203	BA-3-KHA-3456	VIP	30	Available
204	BA-4-KHA-4567	AC	42	Available
205	BA-5-KHA-5678	Tourist	35	Available
206	BA-6-KHA-6789	Sleeper	28	Available
207	BA-7-KHA-7890	Mini	25	Available

```
7 rows selected.
```

Figure 23: Displaying Route Table

17. Add Rows in Staff Table

I. Insert and Commit Command

```
SQL> INSERT INTO Staff VALUES(701, 'Kiran', 'Thapa', '981111001', 'Driver', 'Active');
1 row created.

SQL> INSERT INTO Staff VALUES(702, 'Ramesh', 'Adhikari', '981111002', 'Driver', 'Active');
1 row created.

SQL> INSERT INTO Staff VALUES(703, 'Suresh', 'Karki', '9845115803', 'Conductor', 'Active');
1 row created.

SQL> INSERT INTO Staff VALUES(704, 'Mahesh', 'Shrestha', '9816385624', 'Driver', 'Active');
1 row created.

SQL> INSERT INTO Staff VALUES(705, 'Bina', 'Gurung', '9862847650', 'Conductor', 'Active');
1 row created.

SQL> INSERT INTO Staff VALUES(706, 'Santosh', 'Rai', '9838654706', 'Driver', 'Active');
1 row created.

SQL> INSERT INTO Staff VALUES(707, 'Puja', 'Lama', '9837534707', 'Conductor', 'Active');
1 row created.

SQL> COMMIT;
Commit complete.
```

Figure 24: Inserting Values in Staff Table

II. Select Command

```
SQL> SELECT * FROM Staff;
```

STAFFID	STAFFFIRSTNAME	STAFFLASTNAME	STAFFPHONENUMBE	ROLE	STAFFSTATUS
701	Kiran	Thapa	981111001	Driver	Active
702	Ramesh	Adhikari	981111002	Driver	Active
703	Suresh	Karki	9845115803	Conductor	Active
704	Mahesh	Shrestha	9816385624	Driver	Active
705	Bina	Gurung	9862847650	Conductor	Active
706	Santosh	Rai	9838654706	Driver	Active
707	Puja	Lama	9837534707	Conductor	Active

```
7 rows selected.
```

Figure 25: Displaying Staff Table

18. Add Rows in Trip Table

I. Insert and Commit Command

```
SQL> INSERT INTO Trip VALUES(301, DATE '2025-02-01', '07:00', '14:00', 'Scheduled', 101, 201);
1 row created.

SQL> INSERT INTO Trip VALUES(302, DATE '2025-02-02', '06:30', '12:00', 'Scheduled', 102, 202);
1 row created.

SQL> INSERT INTO Trip VALUES(303, DATE '2025-02-03', '08:00', '15:00', 'Scheduled', 103, 203);
1 row created.

SQL> INSERT INTO Trip VALUES(304, DATE '2025-02-04', '05:30', '16:00', 'Scheduled', 104, 204);
1 row created.

SQL> INSERT INTO Trip VALUES(305, DATE '2025-02-05', '09:00', '13:00', 'Scheduled', 105, 205);
1 row created.

SQL> INSERT INTO Trip VALUES(306, DATE '2025-02-06', '07:15', '14:30', 'Scheduled', 106, 206);
1 row created.

SQL> INSERT INTO Trip VALUES(307, DATE '2025-02-07', '10:00', '11:00', 'Scheduled', 107, 207);
1 row created.

SQL> COMMIT;

Commit complete.
```

Figure 26: Inserting Values in Trip Table

II. Select Command

```
SQL> SELECT * FROM Trip;
```

TRIPID	TRIPDATE	DEPARTURET	ARRIVALTIM	TRIPSTATUS	ROUTEID	BUSID
301	01-FEB-25	07:00	14:00	Scheduled	101	201
302	02-FEB-25	06:30	12:00	Scheduled	102	202
303	03-FEB-25	08:00	15:00	Scheduled	103	203
304	04-FEB-25	05:30	16:00	Scheduled	104	204
305	05-FEB-25	09:00	13:00	Scheduled	105	205
306	06-FEB-25	07:15	14:30	Scheduled	106	206
307	07-FEB-25	10:00	11:00	Scheduled	107	207

```
7 rows selected.
```

Figure 27: Displaying Trip Table

19. Add Rows in Booking Table

I. Insert and Commit Command

```
SQL> INSERT INTO Booking VALUES (401, DATE '2025-01-25', 'Confirmed', 1, 301);
1 row created.

SQL> INSERT INTO Booking VALUES (402, DATE '2025-01-25', 'Confirmed', 2, 302);
1 row created.

SQL> INSERT INTO Booking VALUES (403, DATE '2025-01-26', 'Confirmed', 3, 303);
1 row created.

SQL> INSERT INTO Booking VALUES (404, DATE '2025-01-26', 'Confirmed', 4, 304);
1 row created.

SQL> INSERT INTO Booking VALUES (405, DATE '2025-01-27', 'Confirmed', 5, 305);
1 row created.

SQL> INSERT INTO Booking VALUES (406, DATE '2025-01-27', 'Confirmed', 6, 306);
1 row created.

SQL> INSERT INTO Booking VALUES (407, DATE '2025-01-28', 'Confirmed', 7, 307);
1 row created.

SQL> COMMIT;

Commit complete.
```

Figure 28: Inserting Values in Booking Table

II. Select Command

```
SQL> SELECT * FROM Booking;
```

BOOKINGID	BOOKINGDATE	BOOKINGSTATU	PASSENGERID	TRIPID
410	29-JAN-25	Confirmed	2	303
411	30-JAN-25	Confirmed	2	304
412	29-JAN-25	Confirmed	3	304
413	30-JAN-25	Confirmed	3	305
401	25-JAN-25	Confirmed	1	301
402	25-JAN-25	Confirmed	2	302
403	26-JAN-25	Confirmed	3	303
404	26-JAN-25	Confirmed	4	304
405	27-JAN-25	Confirmed	5	305
406	27-JAN-25	Confirmed	6	306
407	28-JAN-25	Confirmed	7	307
408	29-JAN-25	Confirmed	1	302
409	30-JAN-25	Confirmed	1	303

```
13 rows selected.
```

Figure 29: Displaying Booking Table

20. Add Rows in Ticket Table

I. Insert and Commit Command

```
SQL> INSERT INTO Ticket VALUES (501, 1500, DATE '2025-01-25', 'Issued', 401);
1 row created.

SQL> INSERT INTO Ticket VALUES (502, 1200, DATE '2025-01-25', 'Issued', 402);
1 row created.

SQL> INSERT INTO Ticket VALUES (503, 1800, DATE '2025-01-26', 'Issued', 403);
1 row created.

SQL> INSERT INTO Ticket VALUES (504, 2000, DATE '2025-01-26', 'Issued', 404);
1 row created.

SQL> INSERT INTO Ticket VALUES (505, 1000, DATE '2025-01-27', 'Issued', 405);
1 row created.

SQL> INSERT INTO Ticket VALUES (506, 1600, DATE '2025-01-27', 'Issued', 406);
1 row created.

SQL> INSERT INTO Ticket VALUES (507, 500, DATE '2025-01-28', 'Issued', 407);
1 row created.

SQL> COMMIT;

Commit complete.
```

Figure 30: Inserting Values in Ticket Table

II. Select Command

```
SQL> SELECT * FROM Ticket;
```

TICKETID	FARE	ISSUEDATE	TICKETSTATUS	BOOKINGID
501	1500.00	25-JAN-25	Issued	401
502	1200.00	25-JAN-25	Issued	402
503	1800.00	26-JAN-25	Issued	403
504	2000.00	26-JAN-25	Issued	404
505	1000.00	27-JAN-25	Issued	405
506	1600.00	27-JAN-25	Issued	406
507	500.00	28-JAN-25	Issued	407
508	1200.00	29-JAN-25	Issued	408
509	1800.00	30-JAN-25	Issued	409

```
9 rows selected.
```

Figure 31: Displaying Ticket Table

21. Add Rows in Payment Table

I. Insert and Commit Command

```
SQL> INSERT INTO Payment VALUES (601, 1500, DATE '2025-01-25', 'Cash', 'Paid', 501);
1 row created.
SQL> INSERT INTO Payment VALUES (602, 1200, DATE '2025-01-25', 'Card', 'Paid', 502);
1 row created.
SQL> INSERT INTO Payment VALUES (603, 1800, DATE '2025-01-26', 'Online','Paid', 503);
1 row created.
SQL> INSERT INTO Payment VALUES (604, 2000, DATE '2025-01-26', 'Cash', 'Paid', 504);
1 row created.
SQL> INSERT INTO Payment VALUES (605, 1000, DATE '2025-01-27', 'Card', 'Paid', 505);
1 row created.
SQL> INSERT INTO Payment VALUES (606, 1600, DATE '2025-01-27', 'Online','Paid', 506);
1 row created.
SQL> INSERT INTO Payment VALUES (607, 500, DATE '2025-01-28', 'Cash', 'Paid', 507);
1 row created.
SQL> COMMIT;
Commit complete.
```

Figure 32: Inserting Values in Payment Table

II. Select Command

```
SQL> SELECT * FROM Payment;

PAYMENTID      AMOUNT  PAYMENTDATE  PAYMENTMETHOD  PAYMENTSTATU  TICKETID
-----
          601    1500.00  25-JAN-25      Cash           Paid           501
          602    1200.00  25-JAN-25      Card           Paid           502
          603    1800.00  26-JAN-25     Online         Paid           503
          604    2000.00  26-JAN-25      Cash           Paid           504
          605    1000.00  27-JAN-25      Card           Paid           505
          606    1600.00  27-JAN-25     Online         Paid           506
          607     500.00  28-JAN-25      Cash           Paid           507
          608    1200.00  29-JAN-25      Card           Paid           508
          609    1800.00  30-JAN-25     Online         Paid           509

9 rows selected.
```

Figure 33: Displaying Payment Table

22. Add Rows in Passenger_Trip Table

I. Insert and Commit Command

```
SQL> INSERT INTO Passenger_Trip VALUES (1, 301);
1 row created.
SQL> INSERT INTO Passenger_Trip VALUES (2, 302);
1 row created.
SQL> INSERT INTO Passenger_Trip VALUES (3, 303);
1 row created.
SQL> INSERT INTO Passenger_Trip VALUES (4, 304);
1 row created.
SQL> INSERT INTO Passenger_Trip VALUES (5, 305);
1 row created.
SQL> INSERT INTO Passenger_Trip VALUES (6, 306);
1 row created.
SQL> INSERT INTO Passenger_Trip VALUES (7, 307);
1 row created.
SQL> COMMIT;
Commit complete.
```

Figure 34: Inserting Values in Passenger_Trip Table

II. Select Command

```
SQL> SELECT * FROM Passenger_Trip;

PASSENGERID      TRIPID
-----
          1          301
          2          302
          3          303
          4          304
          5          305
          6          306
          7          307

7 rows selected.
```

Figure 35: Displaying Passenger_Trip Table

6 Database Querying

The Database Querying technique is used for retrieving and processing data compiled in a database. SQL queries are used in database querying. The Information Querying technique is used for selecting data as per the requirements of the user. Transaction Querying technique is used in updating data. The INSERT, UPDATE, and DELETE statements in SQL queries enable the Transaction Querying technique. The user utilizes database querying techniques as these techniques allow the user to access data efficiently and perform database transactions effectively.

6.1 Information Query

6.1.1 Information Query 1: List all routes along with the total number of trips scheduled on each route

```
SQL> SELECT r.RouteID, r.RouteName, COUNT(t.TripID) AS TotalTrips
2  FROM Route r
3  LEFT JOIN Trip t ON r.RouteID = t.RouteID
4  GROUP BY r.RouteID, r.RouteName
5  ORDER BY r.RouteID;
```

ROUTEID	ROUTENAME	TOTALTRIPS
101	Kathmandu-Pokhara	1
102	Kathmandu-Chitwan	1
103	Pokhara-Butwal	1
104	Kathmandu-Dharan	1
105	Butwal-Chitwan	1
106	Lalitpur-Pokhara	1
107	Bhaktapur-KTM	1

7 rows selected.

Figure 36: List all routes along with total number of trips

6.1.2 Information Query 2: List all trips departing from Pokhara with departure time and route name

```
SQL> SELECT t.TripID, t.DepartureTime, r.RouteName
2  FROM Trip t
3  JOIN Route r ON t.RouteID = r.RouteID
4  WHERE r.OriginCity = 'Pokhara'
5  ORDER BY t.DepartureTime;
```

TRIPID	DEPARTURET	ROUTENAME
303	08:00	Pokhara-Butwal

Figure 37: Trips departing from Pokhara

6.1.3 Information Query 3: List passengers whose name starts with 'A' and total trips booked

```
SQL> SELECT p.FirstName, p.LastName, COUNT(b.TripID) AS TotalTrips
2  FROM Passenger p
3  LEFT JOIN Booking b ON p.PassengerID = b.PassengerID
4  WHERE p.FirstName LIKE 'A%'
5  GROUP BY p.FirstName, p.LastName
6  ORDER BY TotalTrips DESC;
```

FIRSTNAME	LASTNAME	TOTALTRIPS
Aakriti	Gurung	1

Figure 38: Passengers whose name starts with A

6.1.4 Information Query 4: Passengers who booked more than 2 trips in December 2025

```
SQL> SELECT p.FirstName, p.LastName, COUNT(b.TripID) AS TripsInDec2025
2 FROM Passenger p
3 JOIN Booking b ON p.PassengerID = b.PassengerID
4 WHERE b.BookingDate BETWEEN DATE '2025-12-01' AND DATE '2025-12-31'
5 GROUP BY p.FirstName, p.LastName
6 HAVING COUNT(b.TripID) > 2
7 ORDER BY TripsInDec2025 DESC;

no rows selected
```

Figure 39: Passengers who booked more than 2 trips in December 2025

As there are no records of passengers who took more than two flights in December of 2025, there were no results from the query because there were no records to match the criteria.

6.1.5 Information Query 5: Tickets issued in January 2025 with payment status

```
SQL> SELECT p.FirstName, p.LastName, t.TripID,
2         NVL(pay.PaymentStatus, 'Unpaid') AS PaymentStatus
3 FROM Ticket ti
4 JOIN Booking b ON ti.BookingID = b.BookingID
5 JOIN Passenger p ON b.PassengerID = p.PassengerID
6 JOIN Trip t ON b.TripID = t.TripID
7 LEFT JOIN Payment pay ON ti.TicketID = pay.TicketID
8 WHERE ti.IssueDate BETWEEN DATE '2025-01-01' AND DATE '2025-01-31'
9 ORDER BY t.TripID;
```

FIRSTNAME	LASTNAME	TRIPID	PAYMENTSTATU
Samikshya	Parajuli	301	Paid
Czan	Thapa	302	Paid
Rebika	Pun	303	Paid
Yoongi	Sunar	304	Paid
Aakriti	Gurung	305	Paid
Newson	Ghimire	306	Paid
Deep	Magar	307	Paid

7 rows selected.

Figure 40: Tickets issued in January 2025 with payment status

6.2 Transaction Query

6.2.1 Transaction Query 1: Find the trip with the latest departure time

```
SQL> SELECT *
  2 FROM Trip
  3 WHERE DepartureTime = (SELECT MAX(DepartureTime) FROM Trip);
```

TRIPID	TRIPDATE	DEPARTURET	ARRIVALTIM	TRIPSTATUS	ROUTEID	BUSID
307	07-FEB-25	10:00	11:00	Scheduled	107	207

Figure 41: The trip with the latest departure time

6.2.2 Transaction Query 2: Top three passengers who booked the most trips

```
SQL> SELECT p.PassengerID,
  2         p.FirstName,
  3         p.LastName,
  4         COUNT(b.TripID) AS TotalTrips
  5 FROM Passenger p
  6 JOIN Booking b ON p.PassengerID = b.PassengerID
  7 GROUP BY p.PassengerID, p.FirstName, p.LastName
  8 HAVING COUNT(b.TripID) >= 3
  9 ORDER BY TotalTrips DESC;
```

PASSENGERID	FIRSTNAME	LASTNAME	TOTALTRIPS
3	Rebika	Pun	3
2	Czan	Thapa	3
1	Samikshya	Parajuli	3

Figure 42: Three passengers who booked the most trips

6.2.3 Transaction Query 3: Passengers who paid more than average payment

```
SQL> SELECT p.FirstName, p.LastName, SUM(pay.Amount) AS TotalPaid
 2  FROM Passenger p
 3  JOIN Booking b ON p.PassengerID = b.PassengerID
 4  JOIN Ticket t ON b.BookingID = t.BookingID
 5  JOIN Payment pay ON t.TicketID = pay.TicketID
 6  GROUP BY p.FirstName, p.LastName
 7  HAVING SUM(pay.Amount) > (SELECT AVG(Amount) FROM Payment)
 8  ORDER BY TotalPaid DESC;
```

FIRSTNAME	LASTNAME	TOTALPAID
Samikshya	Parajuli	4500
Yoongi	Sunar	2000
Rebika	Pun	1800
Newson	Ghimire	1600

Figure 43: Passengers who paid more than average payment

6.2.4 Transaction Query 4: Total revenue generated per route

```
SQL> SELECT r.RouteID, r.RouteName,
 2  SUM(pay.Amount) AS TotalRevenue,
 3  COUNT(DISTINCT t.TripID) AS TotalTrips
 4  FROM Route r
 5  JOIN Trip t ON r.RouteID = t.RouteID
 6  JOIN Booking b ON t.TripID = b.TripID
 7  JOIN Ticket ti ON b.BookingID = ti.BookingID
 8  JOIN Payment pay ON ti.TicketID = pay.TicketID
 9  GROUP BY r.RouteID, r.RouteName
10  ORDER BY TotalRevenue DESC;
```

ROUTEID	ROUTENAME	TOTALREVENUE	TOTALTRIPS
103	Pokhara-Butwal	3600	1
102	Kathmandu-Chitwan	2400	1
104	Kathmandu-Dharan	2000	1
106	Lalitpur-Pokhara	1600	1
101	Kathmandu-Pokhara	1500	1
105	Butwal-Chitwan	1000	1
107	Bhaktapur-KTM	500	1

7 rows selected.

Figure 44: Total revenue generated per route

6.2.5 Transaction Query 5: Trips with less than 50% booking capacity

```

SQL> SELECT t.TripID, t.TripDate, t.DepartureTime, t.ArrivalTime, bu.BusNumber,
2  COUNT(ti.TicketID) AS BookedTickets,
3  bu.SeatingCapacity,
4  ROUND((COUNT(ti.TicketID) / bu.SeatingCapacity) * 100, 2) AS PercentBooked
5  FROM Trip t
6  JOIN Bus bu ON t.BusID = bu.BusID
7  LEFT JOIN Booking b ON t.TripID = b.TripID
8  LEFT JOIN Ticket ti ON b.BookingID = ti.BookingID
9  GROUP BY t.TripID, t.TripDate, t.DepartureTime, t.ArrivalTime, bu.BusNumber, bu.SeatingCapacity
10 HAVING COUNT(ti.TicketID) < (0.5 * bu.SeatingCapacity)
11 ORDER BY PercentBooked ASC;

```

TRIPID	TRIPDATE	DEPARTURET	ARRIVALTIM	BUSNUMBER	BOOKEDTICKETS	SEATINGCAPACITY	PERCENTBOOKED
302	02-FEB-25	06:30	12:00	BA-2-KHA-2345	1	45	2.22
304	04-FEB-25	05:30	16:00	BA-4-KHA-4567	1	42	2.38
301	01-FEB-25	07:00	14:00	BA-1-KHA-1234	1	40	2.5
305	05-FEB-25	09:00	13:00	BA-5-KHA-5678	1	35	2.86
303	03-FEB-25	08:00	15:00	BA-3-KHA-3456	1	30	3.33
306	06-FEB-25	07:15	14:30	BA-6-KHA-6789	1	28	3.57
307	07-FEB-25	10:00	11:00	BA-7-KHA-7890	1	25	4

7 rows selected.

Figure 45: Trips with less than 50% booking capacity

7 Critical Evaluation

7.1 Critical Evaluation of module, its usage and relation with other subjects

CC5051NI is very significant for the development of understanding in students about how any real-world system stores, manages, and utilizes data efficiently. In simple words, this module helped me to learn the entire database development process right from understanding the business requirements to designing ER diagrams, normalizing data, creating tables, and writing SQL queries.

This module is very practical and of great use since the database is employed in practically each field: banking, transportation, health care, e-commerce, etc. From this module, I learned how good database design eliminates data redundancy, avoids inconsistency, and enhances system performance. Concepts such as normalization up to 3NF, primary keys, foreign keys, and constraints showed me how to maintain data integrity. This module is also linked to a wide range of subjects including Programming, System Analysis & Design, and Web Development. Consider the example of programming subjects that make use of databases for storing as well as retrieving information, or system analysis that provides a proper understanding of business rules prior to designing a database. It will also prove useful in future modules like final year projects that are linked to software development.

Overall, the issues covered in this module provide a strong basis for the design of back-end systems and other database-related tasks.

7.2 Critical Assessment of coursework

The course work was a challenge but proved to be an asset in terms of learning database concepts. Creating a full-fledged database system for SajiloYatra was helpful in relating the theoretical concepts with a practical scenario. The task was helpful in enhancing skills to define entities, attributes, and rules in a business system. One of the most powerful aspects of the coursework was the normalization process, whereby I learned how to carry out the normalization of data from UNF to 1NF, then 2NF, and eventually 3NF. This exercise enabled me to appreciate the significance of performing data normalization with the aim of eliminating duplication and avoiding update anomalies. Working on the data dictionary and final ERD was also an exercise that facilitated my ability to pay keen attention to details and grasp the concept of a database. The practical application component with Oracle SQL was greatly appreciated as I was able to apply the skills of creating tables, entering information, and executing a query. The process of entering information and executing transaction queries enhanced my skills with SQL commands.

However, the coursework was time-consuming. For instance, the normalization process coupled with writing SQL queries was time-consuming. Understanding complex relationships between entities in SQL was also difficult. This requires avoiding errors in foreign keys. Despite the challenges, the coursework was helpful for me to acquire problem-solving skills.

In view of the above, I can confidently say that my coursework on databases has improved my knowledge of database systems, and I am well prepared for my future studies.

8 Drop Query and Database Dump File Creation

8.1 Drop Query

i. Drop Passenger Table

```
SQL> DROP TABLE Passenger;  
Table dropped.
```

Figure 46: Dropped Passenger Table

ii. Drop Route Table

```
SQL> DROP TABLE Route;  
Table dropped.
```

Figure 47: Dropped Route Table

iii. Drop Bus Table

```
SQL> DROP TABLE Bus;  
Table dropped.
```

Figure 48: Dropped Bus Table

iv. Drop Staff Table

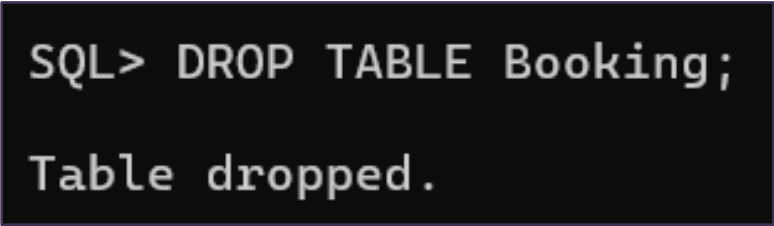
```
SQL> DROP TABLE Staff;  
Table dropped.
```

Figure 49: Dropped Staff Table

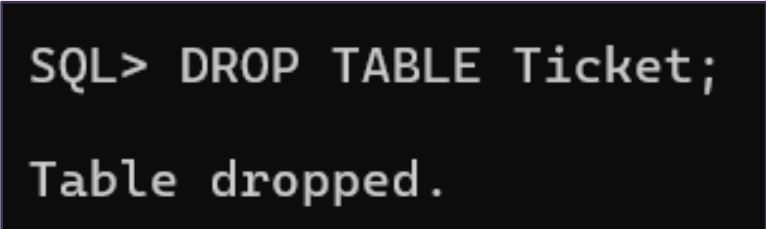
v. Drop Trip Table

```
SQL> DROP TABLE Trip;  
Table dropped.
```

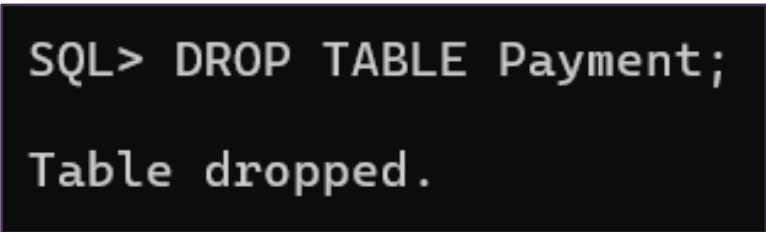
Figure 50: Dropped Trip Table

vi. Drop Booking Table

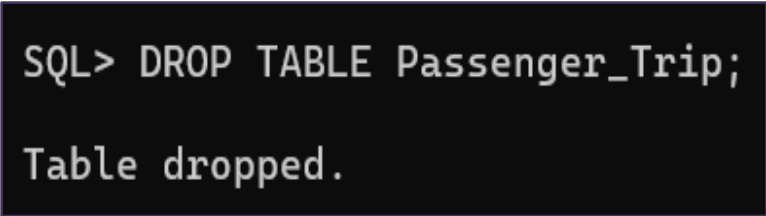
```
SQL> DROP TABLE Booking;  
Table dropped.
```

*Figure 51: Dropped Booking Table***vii. Drop Ticket Table**

```
SQL> DROP TABLE Ticket;  
Table dropped.
```

*Figure 52: Dropped Ticket Table***viii. Drop Payment Table**

```
SQL> DROP TABLE Payment;  
Table dropped.
```

*Figure 53: Dropped Payment Table***ix. Drop Passenger_Trip Table**

```
SQL> DROP TABLE Passenger_Trip;  
Table dropped.
```

Figure 54: Dropped Passenger_Trip Table

8.2 Database Dump File Creation

```

C:\Users\Alina Ghatane>exp alina/24058871 FILE=24058871_Alina_B.K.dmp

Export: Release 11.2.0.2.0 - Production on Sun Jan 18 13:37:53 2026

Copyright (c) 1982, 2009, Oracle and/or its affiliates. All rights reserved.

Connected to: Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production
Export done in WE8MSWIN1252 character set and AL16UTF16 NCHAR character set
server uses AL32UTF8 character set (possible charset conversion)
. exporting pre-schema procedural objects and actions
. exporting foreign function library names for user ALINA
. exporting PUBLIC type synonyms
. exporting private type synonyms
. exporting object type definitions for user ALINA
About to export ALINA's objects ...
. exporting database links
. exporting sequence numbers
. exporting cluster definitions
. about to export ALINA's tables via Conventional Path ...
. . exporting table BOOKING 13 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table BUS 7 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table PASSENGER 7 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table PASSENGER_TRIP 7 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table PAYMENT 9 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table ROUTE 7 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table STAFF 7 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table TICKET 9 rows exported
EXP-00091: Exporting questionable statistics.
. . exporting table TRIP 7 rows exported
EXP-00091: Exporting questionable statistics.
. exporting synonyms
. exporting views
. exporting stored procedures
. exporting operators
. exporting referential integrity constraints
. exporting triggers
. exporting indextypes
. exporting bitmap, functional and extensible indexes
. exporting posttables actions
. exporting materialized views
. exporting snapshot logs
. exporting job queues
. exporting refresh groups and children
. exporting dimensions
. exporting post-schema procedural objects and actions
. exporting statistics
Export terminated successfully with warnings.

```

Figure 55: Drop Query and Database Dump File Creation

9 Conclusion

This coursework has been able to design and develop a central database system for SajiloYatra, a transport service firm functioning in Nepal, to increase the efficiency in booking and management. The coursework has dealt with the whole process of database design, beginning with understanding business requirements and defining business rules, and designing the ER model and normalizing the database up to third normal form (3NF). The proper identification of the entities Passenger, Trip, Route, Bus, Staff, Booking, Ticket, and Payment has assisted in designing a systematic and organized database. The normalization process has eliminated redundancy and assisted in maintaining the integrity and consistency of the data. The practical section in the use of Oracle SQL has assisted in learning the process of table creation, primary and foreign key constraints, and the execution of information and transaction queries. These queries have assisted in learning how the database is used for various normal tasks, such as handling the trip, payment, and profitability process. Although the workload was heavy and stressful, sometimes during the normalization and writing of SQL queries, it greatly improved my theoretical and practical skills concerning databases. In general, the project achieved its goals and gave a very good basis for further academic studies and professional work concerning software and databases.

10 References

Mohtashim, M. (2025). *Introduction of the business and Strategic Strengths*. *Tutorials Point*. Available at: <https://www.tutorialspoint.com/dbms/index.htm> (Accessed 15 September 2025).

Sr, T. J. (2025, August). *Current Business Activities and Operation*. *IBM Think*. Available at: <https://www.ibm.com/think/topics> (Accessed 25 August 2025).

Jain, S. (2025). *The Entity-Relationship Model*. *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/dbms/introduction-of-er-model/> (Accessed 27 November 2025).

Jain, S. (2025). *Normalizations Explained*. *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/> (Accessed 9 October 2025).

(Krishnamurthy, 2025). *SQL Command and Queries Explained*. *Oracle Help Center*. Available at: <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/About-Queries-and-Subqueries.html#GUID-DB7521FE-9329-415E-B583-EA4467E990A7> (Accessed 13 December 2025)

(Thorsell-Arntsen, 2026). *SQL Commands*. *W2Schools*. Available at: https://www.w3schools.com/sql/sql_create_db.asp (Accessed 7 January 2026)

(Wales, 2025). *Database and SQL Explained*. *Wikipedia*. Available at: <https://en.wikipedia.org/wiki/Database> (Accessed 25 December 2025)